Machine Coding: Star Rating

Problem: Build a reusable star rating component: hover to preview, click to set, click again to clear, keyboard accessible, optional half stars and a read-only mode for showing averages like 3.7.
Tech stack: React 19 (Vite, TypeScript, plain CSS)
Time limit: 60 minutes

PART A: Thought process

These steps work for any machine coding task:

1. **Clarify the requirements.** Ask before you build; write the assumptions down.
2. **List the interactions** (what the user does → what they see). This is your checklist.
3. **Sketch the components** and who talks to whom.
4. **Design the state:** the smallest set of values; derive everything else.
5. **Define data shapes and contracts** (props, callbacks, mock data).
6. **Trace one flow end to end** before coding.
7. **Name the hard parts early** so they get time.
8. **Build in layers:** something working on screen first, then add one layer at a time.
9. **Walk through every state** (empty, hover, selected, read-only, edge values).
10. **Wrap up:** edge cases, trade-offs, what you'd add with more time.

Time budget for 60 minutes

Phase	Time
Requirements, questions, assumptions	5 min
Component API + state design	5 min
Core build (render stars, click to set)	12 min
Hover preview + click-again-to-clear	8 min
Keyboard + ARIA	10 min
Half stars + read-only fractional display	10 min
Edge cases, polish, demo page	5 min
Wrap-up and trade-offs	5 min

Tip: think out loud. The interviewer grades how you decide, not only what you type. Say "I'll keep the value controlled by the parent because a form needs to own it" before you write it. If you run out of time, a clear plan for the missing part still earns points.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show N stars (default 5).
- Hovering a star previews the rating up to that star.
- Moving the pointer away restores the selected rating.
- Clicking a star sets the rating; clicking the current rating again clears it to 0.
- Optional half stars: the left half of a star means x.5.
- Read-only mode: shows any value, including fractions like 3.7, and ignores input.
- Keyboard: Tab to focus, arrows change by one step, Home = 0, End = max.

Non-functional requirements

Area	What it means here
Performance	Max 10 stars, so re-rendering all of them on hover is cheap. No memo needed.
Scalability	Many ratings on one page (product list): each instance is independent with only one tiny local state.
Error handling	Values outside 0..max or off the step grid are clamped/snapped; no crash on bad props.
Accessibility	One tab stop, `role="slider"` with `aria-valuenow` / `aria-valuetext`, visible focus ring, read-only exposed as an image with a label, works in high-contrast mode, respects reduced motion.
Usability	Clear hover preview, pointer cursor, text label next to the stars ("Good").
Reusability	Controlled component: `value` + `onChange`, plus `max`, `allowHalf`, `readOnly`, `size`, `label` props.
Extensibility	Pure helper functions for the math, so custom icons (hearts) or RTL can be added without touching the logic.

Clarifying questions

1. Do we need half stars, or whole stars only? Should read-only show exact fractions (3.7)?
2. Can the user clear a rating once set? How (click again, a Clear button)?
3. Is the component controlled by a parent form, or does it own its value?
4. Should hover show a text label ("Excellent") and should the label follow the hover?
5. What keyboard behaviour is expected? Is a screen reader requirement in scope?
6. Any icon requirement (SVG, custom icons) or RTL support?

Assumptions we build with

- 5 stars by default; `max` is configurable.
- Half stars are opt-in through `allowHalf`.

- The component is **controlled**: the parent owns `value` and gets `onChange(newValue)`.
- Clicking the current value clears to 0. The demo also has a Clear button.
- Read-only mode renders any number, including fractions.
- Keyboard uses the slider pattern (one tab stop, arrows step by 1 or 0.5).
- No external icon library: the star shape is a CSS `clip-path`.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page load, value 0	All stars grey, label "Not rated"
Hover the 4th star	Stars 1–4 gold (preview), stored value unchanged
Move the pointer out of the stars	Preview disappears, selected value shown again
Click the 4th star	`onChange(4)`, label "Good"
Click the 4th star again	`onChange(0)`, rating cleared
Half mode: hover left half of star 2	Star 1 full, star 2 half filled (1.5)
Half mode: click right half of star 2	`onChange(2)`
Tab to the component	Focus ring around the whole group
ArrowRight / ArrowUp	Value + 1 step (stops at max)
ArrowLeft / ArrowDown	Value − 1 step (stops at 0)
Home / End	0 / max
Read-only value 3.7	3 full stars, 4th 70 % filled; hover and click do nothing; not focusable
Screen reader on the slider	"Order rating, slider, 4 of 5 stars"
Touch tap on a star	Same as click; preview clears when the finger lifts

---

3. Design the architecture

Module	Responsibility
`App`	Demo page. Owns each rating value, shows the text label and Clear button.
`StarRating`	Renders the stars, handles pointer, click and keyboard, exposes ARIA. Owns only the hover preview.
`utils/rating.ts`	Pure math: how full a star is, which value the pointer points at, clamp/snap for the keyboard.
`StarRating.css`	Star shape (clip-path), fill overlay, focus ring, hover animation.

```
App (value state per rating, label text)
 ├── StarRating  value / onChange / max / allowHalf / readOnly
 │     ├── Star × max   (grey base + gold fill overlay, width = fill %)
 │     └── uses utils/rating.ts
 └── Clear button
```

Start with one component. Split the math into `utils/rating.ts` only once it has three clear functions; that also makes it testable without React. A separate `Star` component is not needed: it is one `span` with one child.

---

4. Design the state / data model

State or field	Purpose
`value` (prop, owned by parent)	The committed rating, 0..max, in steps of 1 or 0.5
`hoverValue: number | null` (local)	The preview while the pointer is over the stars; `null` = no preview
`display` (derived)	`hoverValue ?? value`: what the stars draw
`step` (derived)	`allowHalf ? 0.5 : 1`
`interactive` (derived)	`!readOnly && onChange !== undefined`
fill per star (derived)	`clamp(display - index, 0, 1)` → width in %

```
const [hoverValue, setHoverValue] = useState<number | null>(null);
const step = allowHalf ? 0.5 : 1;
const interactive = !readOnly && onChange !== undefined;
const display = interactive && hoverValue !== null ? hoverValue : value;
```

Senior note. Only one piece of state lives in the component: the hover preview. It is pure UI, nobody outside cares about it, so it stays local. The committed value belongs to whoever submits it (a form, a review page), so it is a prop. Everything else (fill per star, step, the label, ARIA text) is derived on every render. Storing "filled stars" as an array would be a second source of truth that can drift from `value`.

---

5. Design the data structures and contracts

Example data

```
const LABELS = ['Not rated', 'Terrible', 'Bad', 'Okay', 'Good', 'Excellent'];
// index = rating, so LABELS[4] = 'Good'. Only valid for whole-star ratings.

const product = { id: 'p_102', averageRating: 3.7, reviewCount: 1284 };
// read-only display, fractional on purpose
```

Stars are rendered with `Array.from({ length: max })` and the index is the key. That is safe here: the list never reorders, never filters and stars have no own state, so the index is a stable id.

Component contract

```
type StarRatingProps = {
  value: number;                      // 0..max
  onChange?: (value: number) => void; // missing = read-only
  max?: number;                       // default 5
  allowHalf?: boolean;                // default false
  readOnly?: boolean;                 // default false
  size?: number;                      // px, default 32
  label?: string;                     // accessible name, default 'Rating'
};
```

Mock API (if the rating is saved)

```
POST /api/products/p_102/rating
{ "rating": 4 }

200 OK
{ "productId": "p_102", "rating": 4, "averageRating": 3.71, "reviewCount": 1285 }
```

The component does not know about the API. The parent calls it in its `onChange` handler, which keeps the component reusable.

---

6. Trace the data flow

Click on the right half of star 4 (half stars on, current value 2):

```
pointer over star 4 (index 3)
        ↓
onPointerMove → offsetX = clientX − rect.left = 30 of 40px
        ↓
valueFromPointer(3, 30, 40, true) → 4  (right half)
        ↓
setHoverValue(4) → display = 4 → stars 1–4 drawn gold (preview)
        ↓
click → same math → next = 4, next !== value (2)
        ↓
onChange(4) → parent setState(4) → value prop = 4
        ↓
re-render: aria-valuenow = 4, label "4 / 5"
        ↓
pointer leaves the group → setHoverValue(null) → display = value = 4
```

---

7. Important technical challenges

Challenge 1: Hover preview without losing the real value

Problem. The user has rated 2. They hover star 5, then move away without clicking. If hover writes into the same state as the click, the rating silently becomes 5.

Solution. Two values: the committed `value` (prop) and a local `hoverValue`. Draw `hoverValue ?? value`. Clear the preview on `pointerleave` of the **group**, not of each star, so moving between stars does not flicker through the old value.

```
const display = interactive && hoverValue !== null ? hoverValue : value;

<div onPointerLeave={() => setHoverValue(null)}>
  <span onPointerMove={(e) => setHoverValue(pointerValue(e, i))} />
</div>
```

Challenge 2: Half stars and fractional fills with one rendering rule

Problem. Half mode needs 3.5 → three full stars and one half. Read-only needs 3.7 → the 4th star 70 % full. Writing `if half … else if full …` branches per star gets messy, and an icon font cannot show 70 %.

Solution. Every star is a grey base with a gold overlay whose width is `clamp(display − index, 0, 1) × 100 %`. One formula covers 0, 0.5, 1 and any fraction. Both layers sit inside one `clip-path` star, so the overlay is cut to the star shape. For input, the pointer position inside the star picks x.5 or x+1.

```
export function getStarFill(rating: number, index: number) {
  return Math.min(Math.max(rating - index, 0), 1);
}
export function valueFromPointer(index: number, offsetX: number, width: number, allowHalf: boolean) {
  return allowHalf && offsetX < width / 2 ? index + 0.5 : index + 1;
}
```

Challenge 3: Keyboard and screen reader support

Problem. Five clickable `span`s are invisible to keyboard and screen reader users. Making each star a button gives 5 tab stops and the reader says "button" five times without saying the current value.

Solution. Treat the whole group as one `role="slider"`: one tab stop, `aria-valuenow`, and `aria-valuetext` like "4 of 5 stars". Arrows step by 1 or 0.5, Home/End jump, values are clamped and snapped. `preventDefault` stops arrows from scrolling the page. Stars get `aria-hidden`. Read-only becomes `role="img"` with a full label, and is not focusable.

```
const keyMap: Record<string, number> = {
  ArrowRight: value + step, ArrowUp: value + step,
  ArrowLeft: value - step, ArrowDown: value - step,
  Home: 0, End: max,
};
if (!(e.key in keyMap)) return;
e.preventDefault();
onChange?.(clampRating(keyMap[e.key], max, step));
```

Challenge 4: Clicking the current value again clears it

Problem. Once rated, a user cannot go back to "not rated" with a mouse, because the lowest click is 1 (or 0.5).

Solution. In the click handler compare the new value with the current one; if equal, send 0. Keyboard users have Home and Left arrow.

```
const next = pointerValue(e, index);
onChange?.(next === value ? 0 : next);
```

---

8. Implement in the correct order

#	Step	Working result
1	`App` with `useState(0)` and a static row of 5 grey stars	Stars on screen
2	CSS `clip-path` star shape, grey base + gold overlay	Stars look right

#	Step	Working result
3	`getStarFill` + overlay width from `value`	Hard-coded value 3 draws 3 stars
4	`onClick` per star → `onChange(index + 1)`	Click to rate
5	Lift into `StarRating` with `value` / `onChange` / `max` props	Reusable component
6	`hoverValue` + `pointermove` / group `pointerleave`	Hover preview
7	Click same value → 0, plus Clear button in the demo	Clear works
8	`role="slider"`, `tabIndex`, ARIA values, `keydown` map + `clampRating`	Keyboard + screen reader
9	`allowHalf`: `valueFromPointer` with pointer offset, step 0.5	Half stars
10	`readOnly`: no handlers, `role="img"`, fractional fill	Averages like 3.7
11	Focus ring, hover scale with reduced-motion guard, forced-colors fix	Polish
12	Label text with `aria-live`, edge cases, quick self-check of the math	Done

9. Cover all states and edge cases

State / case	What the user sees / how it's handled
Initial (value 0)	All grey, "Not rated", Clear disabled
Hovering	Preview fill up to the pointer; stored value unchanged
Selected	Fill up to `value`, label updates (announced through `aria-live`)
Cleared	Click same star or Clear → 0
Focused	Blue focus ring on the group (only for keyboard, `:focus-visible`)
Read-only	No hover, no click, not in tab order, `role="img"` with label
Fractional value (3.7)	4th star filled 70 %
Value > max or < 0	Fill formula clamps per star, so nothing overflows
Keyboard past the ends	`clampRating` keeps 0..max
Off-grid value by keyboard (3.3 with step 0.5)	Snapped to 3.5
`onChange` not passed	Treated as read-only, no crash
Pointer in the gap between stars	No move event, preview stays on the last star (no flicker)
Touch	Tap fires click with coordinates; `pointerleave` after finger up clears the preview
Reduced motion	Hover scale animation is disabled
Windows high contrast	`forced-color-adjust: none` keeps the star colours visible

10. Final structure

```
_26_star_rating/
├── index.html
├── package.json
├── vite.config.ts
└── src/
    ├── main.tsx
    ├── index.css
    ├── App.tsx                        demo: 3 ratings, labels, Clear
    ├── App.css
    ├── utils/
    │   └── rating.ts                  getStarFill, valueFromPointer, clampRating
    └── components/
        └── StarRating/
            ├── StarRating.tsx
            └── StarRating.css
```

11. Full working code

src/utils/rating.ts

```
// Pure helpers: no React, so they are easy to unit-test.

/** How much of star `index` (0-based) is filled: 0, 0.5, 1, or any fraction for read-only averages
like 3.7. */
export function getStarFill(rating: number, index: number): number {
  return Math.min(Math.max(rating - index, 0), 1);
}

/** Rating under the pointer: the left half of a star means x.5 when half steps are on. */
export function valueFromPointer(index: number, offsetX: number, width: number, allowHalf: boolean):
number {
  return allowHalf && offsetX < width / 2 ? index + 0.5 : index + 1;
}

/** Snap a value to the step grid and keep it inside 0..max (used by the keyboard). */
export function clampRating(value: number, max: number, step: number): number {
  const snapped = Math.round(value / step) * step;
  return Math.min(Math.max(snapped, 0), max);
}
```

src/components/StarRating/StarRating.tsx

```
import { useState, type CSSProperties, type KeyboardEvent, type MouseEvent, type PointerEvent } from
'react';
import { clampRating, getStarFill, valueFromPointer } from '../../utils/rating';
import './StarRating.css';

type StarRatingProps = {
  value: number;
  onChange?: (value: number) => void;
  max?: number;
  allowHalf?: boolean;
  readOnly?: boolean;
  size?: number;
  label?: string;
};

export default function StarRating({
  value,
  onChange,
  max = 5,
  allowHalf = false,
  readOnly = false,
  size = 32,
  label = 'Rating',
}: StarRatingProps) {
  const [hoverValue, setHoverValue] = useState<number | null>(null);
  const step = allowHalf ? 0.5 : 1;
  const interactive = !readOnly && onChange !== undefined;
  // Hover is only a preview; the committed value always comes from the parent.
  const display = interactive && hoverValue !== null ? hoverValue : value;

  const pointerValue = (e: PointerEvent<HTMLSpanElement> | MouseEvent<HTMLSpanElement>, index: number)
=> {
    const rect = e.currentTarget.getBoundingClientRect();
    return valueFromPointer(index, e.clientX - rect.left, rect.width, allowHalf);
  };

  const handleClick = (e: MouseEvent<HTMLSpanElement>, index: number) => {
    const next = pointerValue(e, index);
    // Clicking the current value again clears the rating.
    onChange?.(next === value ? 0 : next);
  };

  const handleKeyDown = (e: KeyboardEvent<HTMLDivElement>) => {
    const keyMap: Record<string, number> = {
      ArrowRight: value + step,
```

```
      ArrowUp: value + step,
      ArrowLeft: value - step,
      ArrowDown: value - step,
      Home: 0,
      End: max,
    };
    if (!(e.key in keyMap)) return;
    e.preventDefault(); // stop arrow keys from scrolling the page
    setHoverValue(null);
    onChange?.(clampRating(keyMap[e.key], max, step));
  };

  const a11yProps = interactive
    ? {
        role: 'slider',
        tabIndex: 0,
        'aria-label': label,
        'aria-valuemin': 0,
        'aria-valuemax': max,
        'aria-valuenow': value,
        'aria-valuetext': `${value} of ${max} stars`,
        onKeyDown: handleKeyDown,
        onPointerLeave: () => setHoverValue(null),
      }
    : { role: 'img', 'aria-label': `${label}: ${value} of ${max} stars` };

  return (
    <div
      className={`star-rating${interactive ? ' is-interactive' : ''}`}
      style={{ '--star-size': `${size}px` } as CSSProperties}
      {...a11yProps}
    >
      {Array.from({ length: max }, (_, i) => (
        <span
          key={i}
          className="star"
          aria-hidden="true"
          onPointerMove={interactive ? (e) => setHoverValue(pointerValue(e, i)) : undefined}
          onClick={interactive ? (e) => handleClick(e, i) : undefined}
        >
          <span className="star-fill" style={{ width: `${getStarFill(display, i) * 100}%` }} />
        </span>
      ))}
    </div>
  );
}
```

src/components/StarRating/StarRating.css

```
.star-rating {
  display: inline-flex;
  gap: 4px;
  border-radius: 6px;
}

.star-rating:focus-visible {
  outline: 2px solid #2563eb;
  outline-offset: 4px;
}

/* One clip-path star: the grey base and the gold fill are both cut to the same shape. */
.star {
  position: relative;
  width: var(--star-size);
  height: var(--star-size);
  background: #d0d5dd;
  clip-path: polygon(50% 0%, 61% 35%, 98% 35%, 68% 57%, 79% 91%, 50% 70%, 21% 91%, 32% 57%, 2% 35%, 39%
35%);
  forced-color-adjust: none; /* keep the colours in Windows high-contrast mode */
}

.star-fill {
  position: absolute;
  inset: 0 auto 0 0;
  background: #f5a524;
}

.is-interactive .star {
  cursor: pointer;
}

@media (prefers-reduced-motion: no-preference) {
  .is-interactive .star {
    transition: transform 120ms ease;
  }
  .is-interactive .star:hover {
    transform: scale(1.15);
  }
}
```

src/App.tsx

```
import { useState } from 'react';
import StarRating from './components/StarRating/StarRating';
import './App.css';

const LABELS = ['Not rated', 'Terrible', 'Bad', 'Okay', 'Good', 'Excellent'];

function App() {
  const [rating, setRating] = useState(0);
  const [halfRating, setHalfRating] = useState(3.5);

  return (
    <main className="app">
      <h1>Star Rating</h1>

      <section className="card">
        <h2>Rate your order</h2>
        <StarRating value={rating} onChange={setRating} label="Order rating" />
        <p className="hint" aria-live="polite">{LABELS[rating]}</p>
        <button type="button" onClick={() => setRating(0)} disabled={rating === 0}>
          Clear
        </button>
      </section>

      <section className="card">
        <h2>Half stars</h2>
        <StarRating value={halfRating} onChange={setHalfRating} allowHalf size={40} label="Movie
rating" />
        <p className="hint" aria-live="polite">{halfRating} / 5</p>
      </section>

      <section className="card">
        <h2>Read-only average</h2>
        <StarRating value={3.7} readOnly size={24} label="Average rating" />
        <p className="hint">3.7 from 1,284 reviews</p>
      </section>
    </main>
  );
}

export default App;
```

src/App.css

```
.app {
  max-width: 480px;
  margin: 0 auto;
  padding: 32px 16px;
}

.card {
  padding: 20px;
  margin-bottom: 16px;
  border: 1px solid #e4e7ec;
  border-radius: 12px;
  background: #fff;
}

.card h2 {
  margin: 0 0 12px;
  font-size: 1rem;
}

.hint {
  margin: 8px 0;
  color: #475467;
}

button {
  padding: 6px 14px;
  border: 1px solid #d0d5dd;
  border-radius: 8px;
  background: #fff;
  font: inherit;
  cursor: pointer;
}

button:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}
```

src/index.css

```
:root {
  font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;
  line-height: 1.5;
  color: #101828;
  background: #f9fafb;
}

body {
  margin: 0;
}
```

src/main.tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

---

12. Wrap-up

Edge cases handled

- Hover never overwrites the committed value; leaving the group restores it.
- Click on the current value clears; keyboard Home / Left reach 0.
- Values are clamped and snapped; out-of-range props never overflow a star.
- Read-only shows exact fractions and is skipped by Tab.
- Missing `onChange` falls back to read-only.
- Arrows do not scroll the page while the rating is focused.
- Reduced motion and high-contrast mode respected.

Trade-offs to mention

Decision	Why	Cost
Controlled component	Parent / form owns the value, easy to submit and reset	Parent must hold state even for a demo
`role="slider"`	One tab stop, announces value, natural for half steps	A radio group (`role="radiogroup"`) reads more like "choose 1 of 5" to some users
CSS `clip-path` star	No assets, any fraction via width, colour with CSS	Shape fixed in CSS; custom icons need an SVG mask instead
Index as key	List never reorders, stars have no state	None here; would be wrong for dynamic lists
No `useCallback` / `memo`	At most 10 stars; the React Compiler is on anyway	None at this size

What to add with more time

- Unit tests for `rating.ts` and React Testing Library tests: hover, click, clear, arrows, read-only.
- `onHoverChange` prop so the parent can show "Excellent" while hovering.
- RTL support (mirror the pointer math and arrow keys when `dir="rtl"`).
- Custom icon prop (SVG mask) and custom colours via CSS variables.
- Form integration: a hidden `<input name>` so it posts with a native form.
- Optimistic save to the API with rollback on error.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you ask whether the component is controlled?

It decides who owns the value. A review form needs to read, reset and submit the rating, so the parent should own it. If the answer had been "standalone widget", I could add a `defaultValue` and keep the value inside.

Q2. What if they want both controlled and uncontrolled usage?

Support `defaultValue` too: keep an internal state that is used only when `value` is `undefined` . `const current = value ?? internalValue` . This is the same pattern native inputs use.

State design

Q3. Why not store `selected` and `hovered` as an array of booleans per star?

Because it duplicates `value` . Every click would have to rebuild the array and a bug can make them disagree. One number plus a formula per star is smaller and cannot go out of sync.

Q4. Why is `hoverValue` local and not in the parent?

Nobody outside needs it; it is pure visual feedback. Lifting it would re-render the parent on every pointer move. If the parent wants a hover label, expose `onHoverChange` instead of moving the state.

Q5. Why `null` for "no hover" instead of `0`?

0 is a valid preview (hovering nothing is not the same as previewing zero). `null` makes "no preview" explicit, so `hoverValue ?? value` works without special cases.

Challenge: hover preview

Q6. Why `pointerleave` on the group and not `mouseleave` on each star?

Per-star leave fires when the pointer crosses from star 3 to star 4, so the preview would flash back to the stored value between stars. On the group it fires once when you really leave. Pointer events also cover touch and pen.

Q7. Why not do the hover effect only in CSS (`:hover ~ .star`)?

The sibling trick works for "fill everything before the hovered one" only with reversed DOM order or `:has()`. It cannot do half stars from the pointer position, and the label text still needs the value in JS. JS state keeps one source of truth.

Challenge: half stars

Q8. How do you know which half was hovered?

`getBoundingClientRect()` on the star gives its left edge and width. `clientX - rect.left` is the offset inside the star. Left of the middle means x.5. The function is pure, so it is tested with numbers, no DOM needed.

Q9. What about RTL languages?

In RTL the first star is on the right. Flip the offset (`width - offsetX`) and swap ArrowLeft/ArrowRight when `dir="rtl"`. The flex row already reverses visually because of `inline-flex`.

Q10. Why not render two half-star buttons per star?

Ten elements and twice the event handlers for the same result. It also doubles the screen reader noise if they are focusable. One element per star plus math is simpler.

Challenge: accessibility

Q11. Why `role="slider"` and not five radio buttons?

Both are valid. The slider gives one tab stop, arrow keys and a spoken value, and fits 0.5 steps well. A native radio group (`<input type="radio">` with labels) gets keyboard for free and posts with a form. I would pick radios if the team prefers native elements and has no half stars.

Q12. How does a screen reader user know the current value?

`aria-valuenow` and `aria-valuetext="4 of 5 stars"`, so it reads "Order rating, slider, 4 of 5 stars". The text label below has `aria-live="polite"` so changes are announced. Stars are `aria-hidden` to avoid five unnamed items.

Q13. Why is read-only `role="img"` and not a disabled slider?

A disabled slider still sounds like a control the user cannot use. A read-only average is a picture of a number, so `role="img"` with "Average rating: 3.7 of 5 stars" is the honest description, and it stays out of the tab order.

Implementation steps

Q14. Why `e.preventDefault()` in the keydown handler?

Without it ArrowDown and ArrowUp scroll the page while the user is changing the rating. We only prevent it for keys we handle, so Tab and other keys still work.

Q15. Is using the index as the React key OK here?

Yes. The star list is a fixed-length range that never reorders, filters or inserts, and stars hold no state. The index is a stable identity in this case.

Q16. Why does the click handler compute the value again instead of using `hoverValue`?

On touch there may be no pointermove before the click, so `hoverValue` can be `null` or stale. Computing from the click coordinates is always correct.

Performance

Q17. What if there are 500 rating widgets on a product list page?

Each one is 5 spans and one small state, so that is fine. Most of them would be read-only, which have no handlers at all. If it ever got heavy, virtualise the list itself, not the stars.

Q18. Do you need `useCallback` or `React.memo`?

Not at this size, re-rendering 5–10 spans is trivial. This project also has the React Compiler enabled, which memoises automatically. I would only add memo after a profiler shows a problem.

Q19. What if `max` is 100?

The rendering still works, but 100 stars is bad UX. I'd ask if they really want a slider or a number input. Technically it is 100 spans, which is cheap.

Trade-offs and "what if"

Q20. What if saving the rating to the API is slow or fails?

Update the UI optimistically in the parent's `onChange`, send the request, and on error roll back to the previous value and show a toast. Disable the widget or ignore clicks while a save is in flight if double submits matter.

Q21. Why `clip-path` and not an SVG or the ★ character?

The ★ glyph changes with the font and cannot be filled 70 %. An SVG works too (use a `<clipPath>` or mask). `clip-path` on a `span` needs no assets, colours come from CSS, and the overlay width gives any fraction.

Q22. How would you test this?

Pure functions with plain asserts (`getStarFill(3.5, 3) === 0.5`). Component tests with React Testing Library: `getByRole('slider')`, fire `keyDown` ArrowRight and check `aria-valuenow`, click a star and check `onChange` was called with 4, click again and expect 0, render `readOnly` and check there is no slider.